



NoSQL & MongoDB

Documentation

NoSQL Concepts · MongoDB Queries · Aggregation Pipeline

Part 1	Part 2	Part 3
NoSQL Intro	MongoDB Basics	Writing Queries

Part 1 — Introduction to NoSQL

Beyond the relational model

What is NoSQL?

NoSQL (Not Only SQL) refers to a family of database systems that move beyond the traditional relational model. They emerged in the early 2000s to handle the growing demands of web-scale applications: massive data volumes, high availability, flexible schemas, and horizontal scalability.

□ **Key idea:** NoSQL does not mean "no SQL at all" — it means "not only SQL". Some NoSQL systems support a query language similar to SQL.

Limitations of the Relational Model

- ▶ Rigid schema — any structural change requires a table migration
- ▶ Vertical scalability only — scaling means upgrading one expensive server
- ▶ Poor fit for non-uniform data with variable attributes
- ▶ Expensive JOINS at large scale across distributed datasets

The Four NoSQL Families

Type	Typical Use Case	Examples
Key-Value	Simple pairs, caching, sessions	Redis, DynamoDB, Memcached
Document	JSON/BSON objects, profiles, catalogs	MongoDB, CouchDB, Firestore
Wide-Column	Logs, time series, IoT data	Cassandra, HBase, Bigtable
Graph	Complex relationships, social networks	Neo4j, ArangoDB, Amazon Neptune

SQL vs NoSQL — When to Use Which?

Prefer SQL when...	Prefer NoSQL when...
Strict ACID transactions	Flexible, evolving schema
Complex relationships / JOINS	High volume, horizontal scaling
Stable, well-defined structure	Nested documents, arrays of objects
Rich SQL reporting	Agile development, fast iterations

Part 2 — MongoDB Basics

Architecture · Compass · Key Concepts

MongoDB Architecture

MongoDB stores data as BSON documents inside collections, grouped in databases. There is no fixed schema — each document in a collection can have different fields.

Term	Meaning
Database	Logical container for collections (e.g. storeDB)
Collection	Group of documents ≈ SQL table (e.g. products)
Document	Single JSON/BSON record ≈ SQL row
Field	Key-value pair inside a document ≈ SQL column
_id	Primary key — auto-generated ObjectId if not provided
Index	Optimisation structure on a field — same concept as SQL

MongoDB Compass — Quick Tour

Compass is the official GUI for MongoDB. Connect to `mongodb://localhost:27017` and use the panels below.

UI Element	Purpose
Left panel	List of databases and collections — click + to create
Documents tab	View, insert, edit and delete documents
FILTER bar	Type a JSON filter and press Find to query documents
Aggregations tab	Build multi-stage aggregation pipelines visually
Indexes tab	Create and manage indexes

Part 3 — Writing MongoDB Queries

Filters · Updates · Aggregation Pipeline

Basic Filter Syntax

All MongoDB queries are expressed as JSON objects. The simplest query matches a field exactly:

```
{ "field": "value" }

// Example — find all products in category Electronics:
{ "category": "Electronics" }
```

Comparison Operators

When a simple equality is not enough, use \$ operators. Syntax: { "field": { "\$operator": value } }

Operator	Meaning	Example
\$eq	Equal to	{ "price": { "\$eq": 100 } }
\$ne	Not equal to	{ "stock": { "\$ne": 0 } }
\$gt	Greater than	{ "price": { "\$gt": 1000 } }
\$gte	Greater than or equal to	{ "price": { "\$gte": 500 } }
\$lt	Less than	{ "price": { "\$lt": 50 } }
\$lte	Less than or equal to	{ "price": { "\$lte": 200 } }
\$in	Value in a list	{ "cat": { "\$in": ["A","B"] } }
\$nin	Value not in a list	{ "cat": { "\$nin": ["C"] } }

Logical Operators

Operator	Meaning	Example
\$and	All conditions must be true	{ "\$and": [{ "price": { "\$gt": 100 } }, { "stock": { "\$gt": 0 } }] }
\$or	At least one condition is true	{ "\$or": [{ "cat": "Gaming" }, { "cat": "Electronics" }] }
\$not	Negates a condition	{ "price": { "\$not": { "\$gt": 500 } } }

Update Operations

Updates require a filter (which documents) and an update operator (what to change):

```
db.collection.updateOne(
  { "filter_field": "value" },
  { "$operator": { "field": newValue } }
```

```
)
```

Operator	Description
\$set	Set a field to a new value
\$unset	Remove a field from the document
\$inc	Increment (or decrement) a numeric field
\$push	Append an element to an array field
\$pull	Remove elements from an array field
\$rename	Rename a field

The Aggregation Pipeline

An aggregation pipeline is a sequence of stages. Each stage transforms the documents produced by the previous one.

```
db.collection.aggregate([
  { $stage1: { ... } }, // filter, match...
  { $stage2: { ... } }, // group, compute...
  { $stage3: { ... } }  // sort, limit...
])
```

Pipeline Stages

Stage	Role
\$match	Filter documents — equivalent to WHERE in SQL
\$group	Group documents by a key and compute aggregates
\$sort	Sort documents (1 = ascending, -1 = descending)
\$project	Select or transform fields to return
\$limit	Limit the number of output documents
\$skip	Skip a number of documents
\$unwind	Deconstruct an array — one document per element
\$lookup	Join another collection — equivalent to SQL JOIN

Accumulators inside \$group

Accumulator	Description
\$sum	Sum of values — use { "\$sum": 1 } to count documents
\$avg	Arithmetic average
\$min	Minimum value in the group
\$max	Maximum value in the group

\$push	Build an array with all values in the group
\$first	First document value in the group

❏ **Important:** Inside pipeline stages, reference a document field with a \$ prefix: "\$price", "\$category". Without \$, MongoDB treats the value as a plain string.

\$group — Full Example

```
// Group products by category, count them, compute average price
{
  "$group": {
    "_id"      : "$category",
    "count"    : { "$sum": 1 },
    "avgPrice" : { "$avg": "$price" }
  }
}
```

\$sort — Example

```
// Sort by price descending, then by name ascending
{ "$sort": { "price": -1, "name": 1 } }
```

Quick Reference Cheat Sheet

Operation	Syntax
Filter equal	<code>{ "name": "Laptop" }</code>
Filter > 1000	<code>{ "price": { "\$gt": 1000 } }</code>
Filter AND	<code>{ "\$and": [{cond1}, {cond2}] }</code>
Update field	<code>{ "\$set": { "price": 99 } }</code>
Increment field	<code>{ "\$inc": { "stock": 5 } }</code>
Average	<code>{ "\$group": { "_id": null, "avg": { "\$avg": "\$price" } } }</code>
Group by field	<code>{ "\$group": { "_id": "\$category", "n": { "\$sum": 1 } } }</code>
Sort descending	<code>{ "\$sort": { "price": -1 } }</code>
Match in pipeline	<code>{ "\$match": { "stock": { "\$gt": 0 } } }</code>

Glossary

Term	Definition
Collection	Set of documents — equivalent to a SQL table
Document	JSON/BSON object stored in a collection — equivalent to a SQL row
Field	Key-value pair in a document — equivalent to a SQL column
<code>_id</code>	Unique identifier auto-generated by MongoDB (ObjectId type)
Pipeline	Sequence of transformation stages in an aggregation
BSON	Binary JSON — MongoDB's internal storage format
Sharding	Distribution of data across multiple servers for scalability
Index	Data structure that speeds up queries on a specific field